

Create a Custom Joystick With Teensy 4.1

By ThirdEarthDesign in Circuits > Microcontrollers  4,155  5

Published on October 8th, 2024 



"You don't need anyone's permission to make something great."

- Massimo Banzi, co-founder of the Arduino project

One of my latest projects is a hybrid gaming rig (*mainly for sim racing and flight sim use*), so I decided to make a few extra bits and pieces for it. One of those extras was a button box, which is effectively just a joystick with lots of buttons.

To make this button box I needed a HID (*Human Interface Device*) compatible micro-controller, this enables the host computer to recognise the device as a game controller. Knowing that the Arduino Leonardo is one such micro-controller it was the first that come to mind, but following a quick bit of research I soon turned my attention to the Teensy 4.1. It's much smaller than the Leonardo, which is clearly a big bonus, but don't let the small size fool you. The Teensy is a very powerful micro-controller that is packed full of features with lots and lots of inputs and outputs. It basically has everything I needed, and more.

The Teensy works with the Arduino IDE and it only requires a quick install of the Teensy add-on to get things up and running. There are lots of useful libraries and example sketches provided as well (*with joysticks included*), so it's pretty easy to hit the ground running.

I could have just used the joystick library provided by Teensy and that would have worked just fine, but it would have meant having some extra axes and other device interfaces that I'm not going to be using. For my button box I only wanted to have the devices, buttons and axes that I need, so I set about creating my own. This meant that I would need to modify the joystick library and create my own USB HID descriptor.

A USB HID descriptor allows the host computer to determine what the capabilities of a HID device are, essentially it lets the host know what type of device it is, be it a keyboard, a mouse, a joystick or whatever. It also lets the host know how many buttons it has, what axes it has, and what other functions it supports.

Creating a USB HID descriptor can be quite daunting, especially if (*like me*) you've never created one before, so this project aims to walk you through the process of creating a custom Joystick, including the custom USB HID descriptor, with a Teensy 4.1.

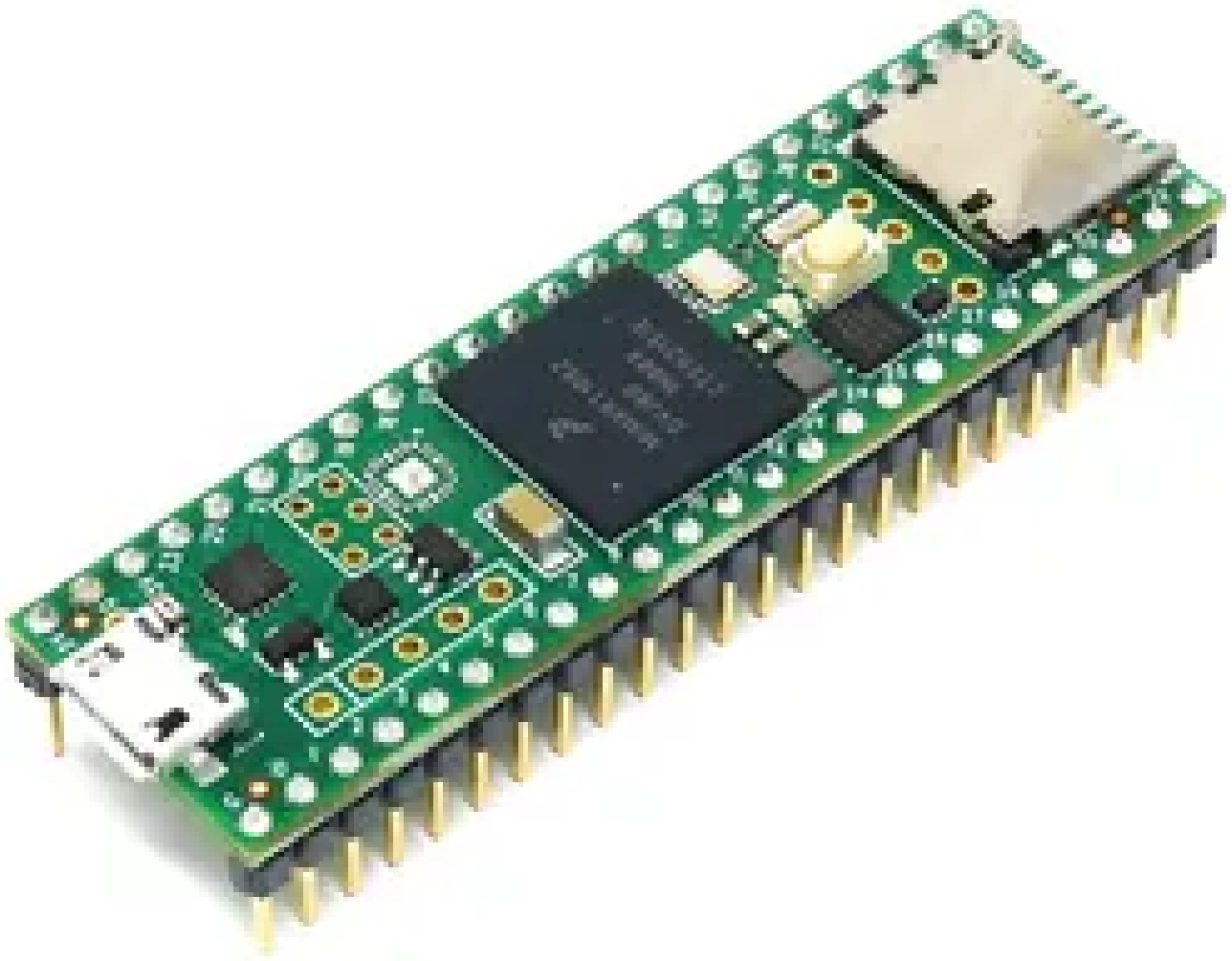
The Teensy joystick library offers two main flavours; a **normal** joystick, which provides 32 buttons, 6 axes and a HAT-switch; *or* the **extreme** joystick, which has 128 buttons, 17 sliders and 4 HAT-switches, on top of the usual X-axis, Y-axis and a couple of Z axes. So lots of possibilities right off the bat it seems.

But within the Teensy libraries it appears that the joystick interfaces are tied-in with other HID devices, so if you want to have a joystick you would have to choose a USB type that also includes some other interfaces, for example '**Serial + Keyboard + Mouse + Joystick**' or '**Flight Sim Controls + Joystick**'. But what if you wanted a stand-alone joystick without that extra stuff? I guess the answer is to create a custom USB type with its own USB HID descriptor. These instructions aim to help you on your path to creating that custom, stand-alone Teensy joystick interface.

The instructions will be broken down into four main steps:

- In **Step 1** we will be adding an additional **USB Type** to the Teensy 4.1 board list, within the Arduino IDE.
- In **Step 2** we will be defining that new USB type in the **usb_desc.h** file.
- **Step 3** will see us creating the USB HID descriptor.
- and in **Step 4** we'll be modifying **usb_joystick.h** so the library knows how to handle the data packets for our controller inputs.

Supplies



To complete this project you'll of course need a Teensy 4.1 microcontroller. They are available direct from the board developer ([PJRC](#)), as well local distributors. I got mine from [ThePiHut](#) here in the U.K.

You'll also need a computer to program your Teensy, and the Arduino IDE with the Teensyduino add-on, as mentioned in the introduction.

Presumably you are reading this Instructable because you are interested in making a custom joystick, so you will probably have some buttons, some rotary

switches, analog inputs or whatever you intend to use for your particular joystick, but in my case I am utilising the following:

- 32x Buttons
 - 13x momentary buttons
 - 3x toggle switches
 - 2x 8-way rotary switches - *with state change / edge detection*
- 3x Axes
 - X, Y *and* Z

Step 1: Add a New USB Type to the Teensy 4.1 Board in the Arduino IDE

These instructions assume that you have already downloaded and installed the Arduino IDE and Teensyduino on your computer. If you haven't, you can follow the instructions provided by PJRC [here](#).

The first step is to modify the Teensyduino boards list to add a new USB type. While it's possible to amend one of the existing USB types to suit our requirements, I feel that adding a new one is more elegant and is less of a 'hack' solution.

Browse to the location where your Arduino Teensy package is installed, in my case this was:

%localAppData%\Arduino15\packages\teensy

Find the file called **boards.txt** and open it in your preferred editor. You should be able to locate the boards file in the following sub-folder:

\hardware\avr\1.59.0

To add the new USB type to the list we need to add the following four lines:

teensy41.menu.usb.customcontroller=Custom Game Controller

teensy41.menu.usb.customcontroller.build.usbtype=USB_CUSTOM_CONTROLLER

teensy41.menu.usb.customcontroller.upload_port.usbtype=USB_CUSTOM_CONTROLLER

teensy41.menu.usb.customcontroller.fake_serial=teensy_gateway

In my boards file I inserted them on lines 128 through 131, so it ended up looking something like this:

```
teensy41.menu.usb.flightsimjoystick=Flight Sim Controls + Joystick
teensy41.menu.usb.flightsimjoystick.build.usbtype=USB_FLIGHTSIM_JOYSTICK
teensy41.menu.usb.flightsimjoystick.upload_port.usbtype=USB_FLIGHTSIM_JOY
STICK
teensy41.menu.usb.flightsimjoystick.fake_serial=teensy_gateway
teensy41.menu.usb.customcontroller=Custom Game Controller
teensy41.menu.usb.customcontroller.build.usbtype=USB_CUSTOM_CONTROLL
ER
teensy41.menu.usb.customcontroller.upload_port.usbtype=USB_CUSTOM_CON
TROLLER
teensy41.menu.usb.customcontroller.fake_serial=teensy_gateway
#teensy41.menu.usb.disable=No USB
#teensy41.menu.usb.disable.build.usbtype=USB_DISABLED
```

If you want to change the descriptive name of the USB type you can amend this in the first line of the additional code as shown below:

```
teensy41.menu.usb.customcontroller=Descriptive name goes here
```

The first step is as easy as that, but in order for the Arduino IDE to reflect the changes to the **boards** file the Arduino cache will need to be cleared. More information can be found on this [here](#), but clearing the cache is just a case of closing the Arduino IDE application and browsing to the following path:

%AppData% then deleting the **arduino-ide** folder. On the next launch of the Arduino IDE app you'll need to select your Teensy 4.1 from the list of available boards again, but you should now be able to see the newly created USB type in the drop down list. Yay!

Step 2: Define the New USB Type

In this step we need to define the new USB type in the **usb_desc.h** file. This is where we can set things like the vendor ID, product ID, the product name and the various other parameters for the new joystick interface.

The **usb_desc.h** is typically located in the path shown below, so browse to it and open the file using your preferred editor:

teensy\hardware\avr\1.59.0\cores\teensy4

The **usb_desc.h** file is made up of an if and **elif** statements, these contain tokens that are referenced during code compilation, dependant upon which USB type is selected. We define our new USB type by adding the following lines of code.

```
#elif defined(USB_CUSTOM_CONTROLLER)
#define VENDOR_ID 0x16C0
#define PRODUCT_ID 0x0488
#define BCD_DEVICE 0x0211
#define MANUFACTURER_NAME {'T','h','i','r','d',' ','E','a','r','t','h',' ','D','e','s','i','g','n'}
#define MANUFACTURER_NAME_LEN 18
#define PRODUCT_NAME {'S','i','m',' ','R','i','g',' ','B','u','t','t','o','n',' ','B','o','x'}
#define PRODUCT_NAME_LEN 18
#define EP0_SIZE 64
#define NUM_ENDPOINTS 3
#define NUM_INTERFACE 2
#define SEREMU_INTERFACE 1 // Serial emulation
#define SEREMU_TX_ENDPOINT 2
#define SEREMU_TX_SIZE 64
#define SEREMU_TX_INTERVAL 1
```

```
#define SEREMU_RX_ENDPOINT 2
#define SEREMU_RX_SIZE 32
#define SEREMU_RX_INTERVAL 2
#define JOYSTICK_INTERFACE 2 // Joystick
#define JOYSTICK_ENDPOINT 3
#define JOYSTICK_SIZE 10 // 10 = custom game controller, 12 = normal joystick,
64 = extreme joystick
#define JOYSTICK_INTERVAL 1
#define ENDPOINT2_CONFIG ENDPOINT_RECEIVE_INTERRUPT +
ENDPOINT_TRANSMIT_INTERRUPT
#define ENDPOINT3_CONFIG ENDPOINT_RECEIVE_UNUSED +
ENDPOINT_TRANSMIT_INTERRUPT
```

You can paste this code in-between any of the existing **#elif** statement blocks, as long as it's after the initial **#if** block and before the first **#endif**. I dropped mine in on line 677 so it followed after the `USB_FLIGHTSIM_JOYSTICK` block.

I'll try to explain what some of these definitions mean, but I won't pretend that I understand it all because I don't. But hopefully there's enough information here to help you make sense of it.

```
#define VENDOR_ID 0x16C0
#define PRODUCT_ID 0x0488
#define BCD_DEVICE 0x0211
```

The Vendor and Product IDs are unique identifiers that are assigned by [USB.org](https://www.usb.org) to developers. 0x16C0 is the vendor ID used by Teensy hardware, 0x0488 is the product ID assigned to 'Teensyduino Flight Sim Controls'. The `BCD_DEVICE` value is a revision release number. Changing these shouldn't prevent the device from working, but it may affect the functionality of the Teensy auto programming

utility, so I'd usually suggest leaving these alone unless you have a specific reason to change them.

```
#define MANUFACTURER_NAME {'T','h','i','r','d',' ','E','a','r','t','h',' ','D','e','s','i','g','n'}  
#define MANUFACTURER_NAME_LEN 18  
#define PRODUCT_NAME {'S','i','m',' ','R','i','g',' ','B','u','t','t','o','n',' ','B','o','x'}  
#define PRODUCT_NAME_LEN 18
```

The manufacturer name and product name are descriptive terms that usually appear in places such as device properties in Windows Device Manager. The product name is also typically used as the controller name within games, so I'd recommend you change this to reflect how you want your particular device to appear. Take care to follow the correct naming format, the length of the names must also be specified, and also note that spaces count as characters in the overall name length as well.

```
#define EP0_SIZE 64  
#define NUM_ENDPOINTS 3  
#define NUM_INTERFACE 2
```

This is where my knowledge gets extra sketchy, so this is just my interpretation or (mis)understanding and should not be considered fact. I believe that the EP0_SIZE specifies the size of the packets. The NUM_ENDPOINTS value specifies the number of endpoints that a device has, endpoint 0 is reserved and I think endpoint 1 may be reserved by the Teensy as well, possibly something to do with the auto programming capability, but I don't really know. Endpoint 2 is assigned to the emulated serial device, while endpoint 3 is assigned to the joystick. There are also two interfaces defined by NUM_INTERFACE, interface 1 being the emulated serial device and interface 2 being the joystick.

```
#define SEREMU_INTERFACE 1 // Serial emulation
```

```
#define SEREMU_TX_ENDPOINT 2
#define SEREMU_TX_SIZE 64
#define SEREMU_TX_INTERVAL 1
#define SEREMU_RX_ENDPOINT 2
#define SEREMU_RX_SIZE 32
#define SEREMU_RX_INTERVAL 2
```

These entries are for the emulated serial device, I guess this is used for the serial monitor output in the Arduino IDE so we'll include this.

```
#define JOYSTICK_INTERFACE 2 // Joystick
#define JOYSTICK_ENDPOINT 3
#define JOYSTICK_SIZE 10 // 10 = custom game controller, 12 = normal joystick,
64 = extreme joystick
#define JOYSTICK_INTERVAL 1
```

These entries are for the joystick interface itself, the most important setting here is the JOYSTICK_SIZE. This defines how many bytes of data are used by the joystick interface, but note that the size specified here will also determine which USB HID descriptor is used when everything is compiled. A joystick size of **12** is the normal joystick descriptor that includes 32 button, 6 axes (*including 2 sliders*) and a POV HAT switch. A size of **64** is the extreme joystick we noted earlier. While **10** is for the custom joystick descriptor that we will be creating in the next step.

```
#define ENDPOINT2_CONFIG ENDPOINT_RECEIVE_INTERRUPT +
ENDPOINT_TRANSMIT_INTERRUPT
#define ENDPOINT3_CONFIG ENDPOINT_RECEIVE_UNUSED +
ENDPOINT_TRANSMIT_INTERRUPT
```

This defines what type of data is sent and received by the various endpoints. As mentioned earlier, endpoint 2 is the emulated serial, while endpoint 3 is the joystick. On the joystick endpoint the receive is defined as UNUSED while the

transmit is an INTERRUPT, I guess this is because a joystick device typically only transmits data.

Step 3: Creating the USB HID Descriptor

Here is where we will define our USB HID descriptor, this is probably the trickiest part to get right. This lets the host know what the HID device is capable of as well as how many bits of data it uses, and what they are for. So let's get busy.

Open the **usb_desc.c** file in your editor. You'll find the file in the same path that we used during the previous step, it just has a slightly different file extension, **.c** instead of **.h**

```
#elif JOYSTICK_SIZE == 10
static uint8_t joystick_report_desc[] = {
0x05, 0x01, // Usage Page (Generic Desktop)
0x09, 0x04, // Usage (Joystick)
0xA1, 0x01, // Collection (Application)
0x15, 0x00, // Logical Minimum (0)
0x25, 0x01, // Logical Maximum (1)
0x75, 0x01, // Report Size (1)
0x95, 0x20, // Report Count (32)
0x05, 0x09, // Usage Page (Button)
0x19, 0x01, // Usage Minimum (Button #1)
0x29, 0x20, // Usage Maximum (Button #32)
0x81, 0x02, // Input (variable,absolute)
0x05, 0x01, // Usage Page (Generic Desktop)
0x09, 0x01, // Usage (Pointer)
0xA1, 0x00, // Collection ( )
0x15, 0x00, // Logical Minimum (0)
0x26, 0xFF, 0x03, // Logical Maximum (1023)
0x75, 0x0A, // Report Size (10)
```

```
0x95, 0x03, // Report Count (3)
0x09, 0x30, // Usage (X)
0x09, 0x31, // Usage (Y)
0x09, 0x32, // Usage (Z)
0x81, 0x02, // Input (variable,absolute)
0xC0, // End Collection
0x75, 0x01, // REPORT_SIZE (1) - for the padded bits
0x95, 0x12, // REPORT_COUNT (18) - 18 padded bits
0x81, 0x03, // INPUT (Cnst,Var,Abs) - constant field for padding
0xC0 // End Collection
};
```

Just like in the previous step this code is one of a number of **elif** statements. You'll need to be sure to drop this particular code somewhere between the `#ifdef JOYSTICK_INTERFACE` and the `#endif // JOYSTICK_SIZE` lines. In my case I placed it after the block of code for `JOYSTICK_SIZE == 12` and before the block for `JOYSTICK_SIZE == 64`, so around line 327.

This block of code is well commented so it is a little more intuitive, the Report Size and the Report Count make reference the number of data bits used, the Logical Minimum and Logical Maximum specify the value range of the actual data, and there are also extra 'empty' data bits added at the end in order to pad the bytes. This is because we have to have complete bytes with a full 8 bits of data in each to properly align them, but there'll be a little bit more on this later on.

While this USB HID descriptor is relatively straight forward some can get rather complicated. There's a whole host of parameters, values and other things to comprehend, so if you want to get a bit more in depth I'd recommend taking a look at some of the following articles, as well as doing your own research:

- [Introduction to HID report descriptors](#) on kernel.org

- [Understanding HID report descriptors](#) by Who-T
- [Custom HID Devices](#) on Adafruit (*part of this is specific to the CircuitPython, but it has some generally relevant information too*)

There are also some software tools that you might find useful, so I'd recommend you take a look at these:

- [HID Descriptor Tool](#) on usb.org (*particularly useful for quickly finding the correct BASE values for HID items*)
- [Microsoft HID Tools](#) on GitHub

Ok, so we're coming back to the data bits and the byte alignment we briefly touched upon above. There will be a tiny bit of math involved, but fortunately it's very basic. Each function of the joystick is represented by bits of data, for example a button is either on or off, so **on** is a **1**, **off** is a **0**. That's 1 bit per button, nice and easy, we have 32 buttons, so that's 32 bits.

We also have 3 axes (*X, Y and Z*). Each axis has a value range from 0 to 1023, where 512 would typically be the center position. These 'analog' values are converted into digital bits via the micro-controller's ADC (*analog to digital converter*), so the more bits you have for a given value range, the higher the positional accuracy will be. The number of bits that are available per axis is generally limited by the number of bits that are supported by the ADC, I believe the Teensy 4.1 supports up to 12-bits, but we're using 10-bits in this particular example. So 3 axes at 10 bits per axis gives us another 30 bits of data, add that to the 32 bits already used by the buttons and we get a grand total of 62 bits.

Now remember what we said earlier, we have to have complete bytes of data that are properly aligned for the data packets to be sent. There are 8 bits for every 1 byte, so if we take our total number of bits (**62**) and divide them by the number of bits in a byte (**8**), we get **62 / 8 = 7.75**, which means we are 2 bits short of having

8 complete bytes. In theory we would only need to add 2 bits to complete the final byte, resulting in a joystick size of 8, but for some reason the sketch won't compile with 8 bytes. So instead we pad 18 bits and bring the total number of bytes to 10. I don't fully understand why it won't compile with a joystick size of 8 but I suspect it has something to do with the code in the joystick library. We'll touch upon that briefly in the next step, but we won't dwell on it too much since everything can work perfectly fine regardless.

Step 4: The Joystick Library

In this final step we'll be concerning ourselves with the **usb_joystick.h** file, this is the Teensy joystick library that handles how the data bits are packaged and sent.

Browse to the same path as we used previously and open the **usb_joystick.h** file in your editor.

```
#elif JOYSTICK_SIZE == 10
void button(uint8_t button, bool val) {
    if (--button >= 32) return;
    if (val) usb_joystick_data[0] |= (1 << button);
    else usb_joystick_data[0] &= ~(1 << button);
    if (!manual_mode) usb_joystick_send();
}

void X(unsigned int val) {
    if (val > 1023) val = 1023;
    usb_joystick_data[1] = (usb_joystick_data[1] & 0xFFFFC00F) | (val << 0);
    if (!manual_mode) usb_joystick_send();
}

void Y(unsigned int val) {
    if (val > 1023) val = 1023;
    usb_joystick_data[1] = (usb_joystick_data[1] & 0xFF003FFF) | (val << 10);
    if (!manual_mode) usb_joystick_send();
}

void position(unsigned int x, unsigned int y) {
    if (x > 1023) x = 1023;
    if (y > 1023) y = 1023;
    usb_joystick_data[1] = (usb_joystick_data[1] & 0xFFF00000)
```

```

| (x << 0) | (y << 10);
if (!manual_mode) usb_joystick_send();
}

void Z(unsigned int val) {
if (val > 1023) val = 1023;
usb_joystick_data[1] = (usb_joystick_data[1] & 0x00FFFFFF) | (val << 20);
if (!manual_mode) usb_joystick_send();
}

```

As we did in the previous step we are again going to drop in some code that is specific to our new custom joystick size of 10. I put this block of code in at line 124, between the **elif** statements for JOYSTICK_SIZE == 12 and JOYSTICK_SIZE == 64. This code was based on what was being used by JOYSTICK_SIZE == 12, but with two main differences. Firstly the voids that reference the unused axes have been removed (*Z-rotate, the three sliders and the HAT switch*). Then the voids for the buttons and axes that we using need to have the bits shifted to properly reflect the data bits used in the HID descriptor. Pay particular attention to the following four lines of code:

```

usb_joystick_data[1] = (usb_joystick_data[1] & 0xFFFFC00F) | (val << 0);
usb_joystick_data[1] = (usb_joystick_data[1] & 0xFF003FFF) | (val << 10);
| (x << 0) | (y << 10);
usb_joystick_data[1] = (usb_joystick_data[1] & 0x00FFFFFF) | (val << 20);

```

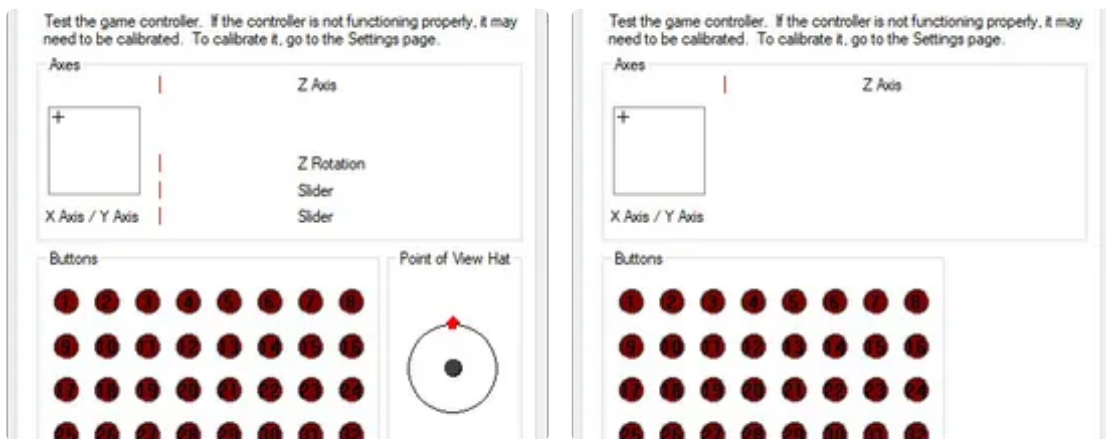
By comparing the lines of code above taken from JOYSTICK_SIZE == 10 with their equivalent values in JOYSTICK_SIZE == 12, while also cross referencing them with the allocated bits in the HID descriptor file, you should hopefully find a logical connection and a rationale to why these changes were necessary to get everything working as expected. I guess you can consider that your homework assignment for the day.

Coming back to what we briefly mentioned at the end of **Step 3**, we are using a joystick size of 10 despite only actually needing 8 bytes to complete the joystick interface. As I said, for some reason the sketch won't compile with the joystick size set to 8, and while I haven't tried too hard to understand the cause, I believe it is related to the formula that's in line 46 of the **usb_joystick.h** file:

```
extern uint32_t usb_joystick_data[(JOYSTICK_SIZE+3)/4];
```

Since the workaround of using additional padding bits will work perfectly fine with no ill effects, I won't lose any sleep over it. But I just thought I'd explain why we are using `JOYSTICK_SIZE == 10` here.

Step 5: And Finally..



Hopefully after following the steps above your custom joystick will be good to go, now you just have to write your Arduino sketch or use one of the Teensy example sketches.

So get coding, get compiling, and upload your sketch to your Teensy. Good luck!

An example Arduino sketch for this project is available to download
JoystickExample.ino

This is a modified version of the 'Complete USB Joystick Example' sketch that's included with Teensyduino.

It has been modified to include arrays for specifying the Teensy input pins used by the buttons and switches, it utilises state change detection for up to two 8-way rotary switches, and the unused axes and HAT switch have been removed.

Note: *If like me you've modified the **usb_desc.h** file to change the name of a device or you've changed something in the **usb_desc.c** file, but Windows isn't reflecting the changes because it 'remembers' the previously connected device, see this [Teensy forum article](#) for tips on how to remove remembered devices from Windows.*

